Docker files /Jenkins/cicd/ important files

Docker lo Kubernetes laga chala .yaml files undavu. **Main ga** Dockerfile **important**, plus Docker Compose use chesthe docker-compose.yml / compose.yaml important.

File names — meaning

- Dockerfile — Docker image build cheyyadaniki
- docker-compose.yml — Multiple containers ni together run/manage cheyyadaniki
- compose.yaml — Docker Compose configuration kosam
- .dockerignore — Docker image build time lo unwanted files exclude cheyyadaniki
- docker-compose.override.yml — Development/local Docker Compose settings override cheyyadaniki

---

1. Dockerfile >> Docker image build cheyyadaniki

```
FROM node:20

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "start"]
```

explanation:

FROM node:20 > base image
→ Docker image build cheyyadaniki starting/base image enti ani chepthundi.
→ Ikkada Node.js 20 image ni base ga use chestunnam.

WORKDIR /app > working directory
→ Container lopala application work cheyyalsina directory ni /app ga set chestundi.

COPY package*.json ./ > files copy
→ package.json and related package files ni container lo /app ki copy chestundi.

RUN npm install > command run cheyyali
→ Application ki kavalsina Node.js dependencies install chestundi.

COPY . . > application files copy
→ Mana current project files anni container lo copy chestundi.

EXPOSE 3000 > application port
→ Application container lo port 3000 use chestundi ani Docker ki declare chestundi.

CMD ["npm", "start"] > startup command
→ Container start ayinappudu npm start command run chestundi.

---

2. docker-compose.yml >> Multiple containers together run cheyyadaniki

```
services:
  netflix:
    build: .
    ports:
      - "3000:3000"
    environment:
      APP_ENV: production

  database:
    image: mysql:8
```

```
  environment:
    MYSQL_ROOT_PASSWORD: example-password
```

explanation:

services: > services/containers
→ Application lo run cheyyalsina containers/services ni ikkada define chestham.

netflix: > service peru
→ Mana application service ki netflix ane peru pettam.

build: . > Docker image build
→ Current directory lo unna Dockerfile use chesi image build cheyyali ani chepthundi.

ports: > port mapping
→ Container port ni host machine port tho connect chestundi.

"3000:3000" > host:container port
→ Host machine port 3000 → Container port 3000 ki connect avtundi.

environment: > environment variables
→ Container ki configuration values provide cheyyadaniki use chestham.

APP_ENV: production > configuration value
→ Application environment production ani container ki provide chestundi.

database: > service peru
→ Database kosam separate service define chestunnam.

image: mysql:8 > Docker image
→ Database container lo MySQL 8 image run avtundi.

MYSQL_ROOT_PASSWORD: example-password > database password
→ MySQL root user ki password provide chestundi.

---

3. .dockerignore >> Unwanted files exclude cheyyadaniki

```
node_modules
.git
.env
*.log
Dockerfile
```

explanation:

node_modules > dependency folder
→ Local node_modules Docker image loki copy kakunda exclude chestundi.

.git > Git files
→ Git repository information image loki vellakunda exclude chestundi.

.env > environment file
→ Sensitive .env file image build context lo include kakunda exclude chestundi.

*.log > log files
→ Anni .log files exclude chestundi.

Dockerfile > Docker build file
→ Dockerfile ni build context nunchi exclude chestundi.

Jenkins files

Jenkins lo **main ga 2 types** gurthupettuko:

- Jenkinsfile — Jenkins CI/CD pipeline define cheyyadaniki
- jenkins.yaml / jenkins.yml — Jenkins Configuration as Code (JCasC) use chesthe configuration kosam

File names — meaning

- Jenkinsfile — CI/CD pipeline define cheyyadaniki
- jenkins.yaml — Jenkins configuration as code kosam
- plugins.txt — Jenkins plugins list maintain cheyyadaniki
- Dockerfile — Jenkins custom Docker image build cheyyadaniki

---

4. Jenkinsfile >> CI/CD pipeline define cheyyadaniki

```
pipeline {
  agent any

  stages {

    stage('Build') {
      steps {
        sh 'npm install'
      }
    }

    stage('Test') {
      steps {
        sh 'npm test'
      }
    }

    stage('Docker Build') {
      steps {
        sh 'docker build -t mycompany/netflix:v1 .'
      }
    }

    stage('Deploy') {
      steps {
        sh 'kubectl apply -f deployment.yaml'
      }
    }
  }
}
```

explanation:

pipeline { > pipeline definition
→ Jenkins lo complete CI/CD pipeline ikkada define chestham.

agent any > execution agent
→ Pipeline stages ni available unna Jenkins agent meeda run cheyyachu ani chepthundi.

stages { > pipeline stages
→ Pipeline lo different steps/stages ikkada define chestham.

stage('Build') > build stage
→ Application build/install process kosam ee stage.

steps { > commands
→ Ee stage lo execute cheyyalsina commands ikkada untayi.

sh 'npm install' > command
→ Application dependencies install chestundi.

stage('Test') > testing stage
→ Application tests run cheyyadaniki ee stage.

sh 'npm test' > test command
→ Application tests execute chestundi.

stage('Docker Build') > Docker image build stage
→ Application kosam Docker image build cheyyadaniki ee stage.

sh 'docker build -t mycompany/netflix:v1 .' > Docker build command
→ Dockerfile use chesi mycompany/netflix:v1 image build chestundi.

stage('Deploy') > deployment stage
→ Application ni Kubernetes lo deploy cheyyadaniki ee stage.

sh 'kubectl apply -f deployment.yaml' > Kubernetes command
→ deployment.yaml ni Kubernetes ki apply chesi application deployment create/update chestundi.

---

5. jenkins.yaml >> Jenkins configuration as code kosam

```
jenkins:
  systemMessage: "Netflix Jenkins Server"

  numExecutors: 2

  securityRealm:
    local:
      allowsSignup: false

unclassified:
  location:
    url: https://jenkins.example.com/
```

explanation:

jenkins: > Jenkins configuration
→ Jenkins server ki sambandhinchina configuration ikkada define chestham.

systemMessage: > Jenkins message
→ Jenkins UI lo display ayye system message ni set chestundi.

numExecutors: 2 > executors count
→ Jenkins same time lo 2 tasks/jobs run cheyyagaladu.

securityRealm: > authentication settings
→ Jenkins users login/authentication ela work avvali ane settings ikkada define chestham.

local: > local authentication
→ Jenkins own local user database use cheyyali ani chepthundi.

allowsSignup: false > user signup rule
→ New users self-signup cheyyakudadhu ani chepthundi.

unclassified: > other Jenkins settings
→ Standard security/job settings kakunda other Jenkins configurations ikkada define chestham.

location: > Jenkins location
→ Jenkins server URL information ikkada define chestham.

url: https://jenkins.example.com/ > Jenkins URL
→ Jenkins server access cheyyalsina URL ni chepthundi.

---

6. plugins.txt >> Jenkins plugins list kosam

```
workflow-aggregator
git
docker-workflow
kubernetes
credentials-binding
```

explanation:

workflow-aggregator > Pipeline plugin
→ Jenkins Pipeline functionality kosam plugin.

git > Git plugin
→ Jenkins Git repositories tho work cheyyadaniki plugin.

docker-workflow > Docker plugin
→ Jenkins pipeline nunchi Docker operations cheyyadaniki plugin.

kubernetes > Kubernetes plugin
→ Jenkins Kubernetes tho integrate avvadaniki plugin.

credentials-binding > credentials plugin
→ Pipeline lo credentials ni securely use cheyyadaniki plugin.

CI/CD lo inka important files
Real-time project structure lo veetini kuda chusthav:

- package.json — Node.js project dependencies/scripts kosam
- package-lock.json — Exact dependency versions lock cheyyadaniki
- pom.xml — Java Maven project configuration kosam
- build.gradle — Java/Gradle build configuration kosam
- requirements.txt — Python dependencies kosam
- .gitignore — Git ki unwanted files exclude cheyyadaniki
- .env — Local environment variables kosam
- Makefile — Common build/deploy commands simplify cheyyadaniki
- sonar-project.properties — SonarQube code analysis configuration kosam
- terraform/*.tf — Infrastructure as Code kosam

---

1. package.json >> Node.js project dependencies/scripts kosam

```
{
  "name": "netflix",
  "version": "1.0.0",
  "scripts": {
    "start": "node app.js",
    "test": "jest"
  },
  "dependencies": {
    "express": "^4.18.0"
  },
  "devDependencies": {
    "jest": "^29.0.0"
  }
}
```

explanation:

name: netflix > project peru
→ Node.js project ki netflix ane peru pettam.

version: 1.0.0 > project version
→ Project current version 1.0.0 ani chepthundi.

scripts: > commands/scripts
→ Project lo frequently run chese commands ikkada define chestham.

start: node app.js > application start command
→ npm start run chesthe node app.js command execute avtundi.

test: jest > testing command
→ npm test run chesthe Jest tests execute avtayi.

dependencies: > application dependencies
→ Application run avvadaniki required packages ikkada untayi.

express: ^4.18.0 > package/version
→ Application ki Express package version 4.18.0 or compatible version kavali.

devDependencies: > development dependencies
→ Development/testing time lo required packages ikkada untayi.

jest: ^29.0.0 > testing package
→ Application tests run cheyyadaniki Jest package use chestunnam.

---

2. package-lock.json >> Exact dependency versions lock cheyyadaniki

```
{
  "name": "netflix",
  "version": "1.0.0",
  "lockfileVersion": 3,
  "packages": {
    "": {
      "name": "netflix",
      "version": "1.0.0"
    },
    "node_modules/express": {
      "version": "4.18.2"
    }
  }
}
```

explanation:

name: netflix > project peru
→ Ee lock file ye project ki belong avtundo chepthundi.

version: 1.0.0 > project version
→ Project version ni chepthundi.

lockfileVersion: 3 > lock file version
→ Ee package-lock file ye lock format version use chestundo chepthundi.

packages: > installed packages
→ Project lo install ayye packages information ikkada maintain chestundi.

"": > main project
→ Empty package name ante main project package ni represent chestundi.

name: netflix > project peru
→ Main project peru netflix.

version: 1.0.0 > project version
→ Main project version 1.0.0.

node_modules/express > package location
→ express package installed location ni represent chestundi.

version: 4.18.2 > exact package version
→ Install cheyyalsina exact Express version 4.18.2 ani lock chestundi.

---

3. pom.xml >> Java Maven project configuration kosam

```
<project>
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.mycompany</groupId>
  <artifactId>netflix</artifactId>
  <version>1.0.0</version>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
      <version>3.2.0</version>
    </dependency>
  </dependencies>
</project>
```

explanation:

project > Maven project
→ Idi complete Maven project configuration ni represent chestundi.

modelVersion: 4.0.0 > Maven model version
→ Maven project structure ye model version follow avtundo chepthundi.

groupId: com.mycompany > project group
→ Project ye organization/group ki belong avtundo identify chestundi.

artifactId: netflix > project name
→ Maven project ki netflix ane artifact name istundi.

version: 1.0.0 > project version
→ Project current version 1.0.0.

dependencies: > required libraries
→ Java application ki kavalsina external libraries ikkada define chestham.

dependency > one dependency
→ Oka required library configuration start avtundi.

groupId: org.springframework.boot > dependency group
→ Spring Boot library ye group ki belong avtundo chepthundi.

artifactId: spring-boot-starter-web > dependency name
→ Web application kosam Spring Boot Web dependency use chestunnam.

version: 3.2.0 > dependency version
→ Spring Boot dependency version 3.2.0.

---

4. build.gradle >> Java Gradle build configuration kosam

```
plugins {
  id 'java'
  id 'org.springframework.boot' version '3.2.0'
}

group = 'com.mycompany'
version = '1.0.0'

repositories {
  mavenCentral()
}

dependencies {
  implementation 'org.springframework.boot:spring-boot-starter-web:3.2.0'
  testImplementation 'org.springframework.boot:spring-boot-starter-test:3.2.0'
}
```

explanation:

plugins: > build plugins
→ Gradle project ki required plugins ikkada define chestham.

id 'java' > Java plugin
→ Project Java application ani Gradle ki chepthundi.

id 'org.springframework.boot' > Spring Boot plugin
→ Project lo Spring Boot functionality use chestunnam.

version '3.2.0' > plugin version
→ Spring Boot plugin version 3.2.0.

group: com.mycompany > project group
→ Project organization/group identify chestundi.

version: 1.0.0 > project version
→ Project version 1.0.0.

repositories: > package repositories
→ Dependencies ekkadi nunchi download cheyyalo chepthundi.

mavenCentral() > repository
→ Maven Central repository nunchi dependencies download chestundi.

dependencies: > required libraries
→ Project ki kavalsina libraries ikkada define chestham.

implementation > application dependency
→ Application run/build kosam required dependency.

testImplementation > testing dependency
→ Tests run cheyyadaniki required dependency.

---

5. requirements.txt >> Python dependencies kosam

```
flask==3.0.0
requests==2.31.0
gunicorn==21.2.0
pytest==7.4.0
```

explanation:

flask==3.0.0 > Python package
→ Python web application kosam Flask 3.0.0 version install chestundi.

requests==2.31.0 > HTTP package
→ APIs/HTTP requests send cheyyadaniki Requests package install chestundi.

gunicorn==21.2.0 > application server
→ Python application ni production lo run cheyyadaniki Gunicorn install chestundi.

pytest==7.4.0 > testing package
→ Python application tests run cheyyadaniki Pytest install chestundi.

---

6. .gitignore >> Git ki unwanted files exclude cheyyadaniki

```
node_modules/
.env
*.log
dist/
build/
.terraform/
```

explanation:

node_modules/ > dependency folder
→ Node.js dependencies Git repository lo commit kakunda exclude chestundi.

.env > environment file
→ Environment variables/sensitive values unna .env file ni Git nunchi exclude chestundi.

*.log > log files
→ Anni log files Git ki track kakunda exclude chestundi.

dist/ > build output
→ Generated distribution files ni Git nunchi exclude chestundi.

build/ > build folder
→ Build output folder ni Git nunchi exclude chestundi.

.terraform/ > Terraform files
→ Terraform generated local files ni Git nunchi exclude chestundi.

---

7. .env >> Environment variables kosam

```
APP_ENV=production
PORT=3000
DB_HOST=localhost
DB_USERNAME=netflix
DB_PASSWORD=example-password
API_URL=https://api.example.com
```

explanation:

APP_ENV=production > application environment
→ Application production environment lo run avtundi ani value provide chestundi.

PORT=3000 > application port
→ Application port 3000 ani chepthundi.

DB_HOST=localhost > database host
→ Database ekkada run avtundo host information provide chestundi.

DB_USERNAME=netflix > database username
→ Database login kosam username provide chestundi.

DB_PASSWORD=example-password > database password
→ Database login kosam password provide chestundi.

API_URL=https://api.example.com > API URL
→ Application use cheyyalsina backend API address provide chestundi.

---

8. Makefile >> Common commands simplify cheyyadaniki

```
install:
	npm install

test:
	npm test

build:
	docker build -t mycompany/netflix:v1 .

deploy:
	kubectl apply -f deployment.yaml
```

explanation:

install: > install command
→ Dependencies install cheyyadaniki install target create chestundi.

npm install > dependency command
→ Project dependencies install chestundi.

test: > testing command
→ Application tests run cheyyadaniki target.

npm test > test command
→ Application tests execute chestundi.

build: > build command
→ Docker image build cheyyadaniki target.

docker build -t mycompany/netflix:v1 . > Docker build command
→ Dockerfile use chesi mycompany/netflix:v1 image build chestundi.

deploy: > deployment command
→ Kubernetes application deploy cheyyadaniki target.

kubectl apply -f deployment.yaml > Kubernetes command
→ deployment.yaml ni Kubernetes cluster ki apply chestundi.

---

9. sonar-project.properties >> SonarQube code analysis kosam

```
sonar.projectKey=netflix
sonar.projectName=Netflix
sonar.sources=src
sonar.tests=test
sonar.host.url=http://sonarqube:9000
```

explanation:

sonar.projectKey=netflix > project key
→ SonarQube lo project ni identify cheyyadaniki unique key.

sonar.projectName=Netflix > project name
→ SonarQube dashboard lo display ayye project name.

sonar.sources=src > source code location
→ Code analysis cheyyalsina source files src folder lo unnayi ani chepthundi.

sonar.tests=test > test location
→ Test files test folder lo unnayi ani chepthundi.

sonar.host.url=http://sonarqube:9000 > SonarQube server URL
→ Code analysis results upload cheyyalsina SonarQube server address.

---

10. terraform/main.tf >> Infrastructure as Code kosam

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}

resource "aws_instance" "netflix" {
  ami           = "ami-xxxxxxxx"
  instance_type = "t3.micro"

  tags = {
    Name = "netflix-server"
  }
}
```

explanation:

terraform { > Terraform configuration
→ Terraform project ki required configuration ikkada define chestham.

required_providers: > required providers
→ Infrastructure create cheyyadaniki kavalsina providers ikkada define chestham.

aws: > AWS provider
→ AWS infrastructure ni Terraform manage cheyyadaniki AWS provider use chestunnam.

source: hashicorp/aws > provider source
→ AWS provider ekkadi nunchi use cheyyalo chepthundi.

version: ~> 5.0 > provider version
→ AWS provider version 5.x compatible versions use cheyyali ani chepthundi.

provider "aws" > AWS configuration
→ AWS provider ki sambandhinchina settings ikkada define chestham.

region: ap-south-1 > AWS region
→ Resources AWS Mumbai region lo create cheyyali ani chepthundi.

resource "aws_instance" "netflix" > infrastructure resource
→ AWS lo EC2 instance create cheyyali ani chepthundi.
ami: ami-xxxxxxxx > machine image
→ EC2 instance ki use cheyyalsina AMI image identify chestundi.
instance_type: t3.micro > server size
→ EC2 instance ki t3.micro machine type use chestunnam.
tags: > resource tags
→ AWS resource ki identifying tags ikkada define chestham.
Name: netflix-server > resource name tag
→ EC2 instance ki netflix-server ane tag/name istundi.

Terraform lo multiple .tf **files kuda undachu**

```
terraform/
├── main.tf        — Main infrastructure resources kosam
├── variables.tf   — Input variables kosam
├── outputs.tf     — Terraform output values kosam
├── providers.tf   — Providers configuration kosam
└── terraform.tfvars — Variable values kosam
```

Important: Terraform lo terraform/*.tf ante **oka single file kaadu**. terraform folder lo multiple .tf files undachu; Terraform vatini kalipi oka configuration laga process chestundi.

1. **ChatGPT said:**

Sure boss. **Terraform lo remaining important** .tf **files** ni same style lo continue chestunna.

11. terraform/variables.tf >> Input variables kosam

```
variable "aws_region" {
 description = "AWS region"
 type      = string
 default    = "ap-south-1"
}

variable "instance_type" {
 description = "EC2 instance type"
 type      = string
 default    = "t3.micro"
}

variable "server_name" {
 description = "Server name"
 type      = string
 default    = "netflix-server"
}
```

explanation:

variable "aws_region" > variable name
→ AWS region value ni variable ga define chestunnam.

description: "AWS region" > variable information
→ Ee variable denikosam use chestunnamo chepthundi.

type: string > variable type
→ Ee variable lo text/string value untundi ani chepthundi.

default: "ap-south-1" > default value
→ User value ivvakapothe ap-south-1 automatically use avtundi.

variable "instance_type" > variable name
→ EC2 instance type ni variable ga define chestunnam.

description: "EC2 instance type" > variable information
→ Ee variable EC2 machine type kosam ani chepthundi.

type: string > variable type
→ Value string/text format lo untundi.

default: "t3.micro" > default value
→ Default EC2 instance type t3.micro.

variable "server_name" > variable name
→ Server name ni variable ga define chestunnam.

description: "Server name" > variable information
→ Ee variable server name kosam ani chepthundi.

type: string > variable type
→ Server name text/string value.

default: "netflix-server" > default value
→ Default server name netflix-server.

---

12. terraform/outputs.tf >> Terraform output values kosam

```
output "server_id" {

 description = "EC2 instance ID"

 value      = aws_instance.netflix.id

}


output "server_name" {

 description = "EC2 server name"

 value      = aws_instance.netflix.tags["Name"]

}
```

explanation:

output "server_id" > output name
→ Terraform apply ayyaka manaki server ID output ga kavali ani chepthundi.

description: "EC2 instance ID" > output information
→ Ee output EC2 instance ID ani explain chestundi.

value: aws_instance.netflix.id > output value
→ Create ayina Netflix EC2 instance ID ni output chestundi.

output "server_name" > output name
→ Server name ni output ga chupinchadaniki define chestunnam.

description: "EC2 server name" > output information
→ Ee output server name kosam ani chepthundi.

value: aws_instance.netflix.tags["Name"] > output value
→ EC2 instance lo unna Name tag value ni output chestundi.

---

13. terraform/providers.tf >> Terraform providers configuration kosam

```
terraform {

 required_providers {

   aws = {

    source  = "hashicorp/aws"

    version = "~> 5.0"

   }

 }

}
```

```
provider "aws" {

  region = var.aws_region

}
```

explanation:

terraform { > Terraform configuration
→ Terraform project ki required provider settings ikkada define chestham.

required_providers: > required providers
→ Terraform ki ye providers kavalo chepthundi.

aws: > AWS provider
→ AWS infrastructure ni manage cheyyadaniki AWS provider use chestunnam.

source: hashicorp/aws > provider source
→ AWS provider source hashicorp/aws ani chepthundi.

version: ~> 5.0 > provider version
→ AWS provider lo compatible 5.x version use cheyyali ani chepthundi.

provider "aws" > AWS provider configuration
→ AWS provider ki sambandhinchina settings ikkada define chestham.

region: var.aws_region > AWS region
→ AWS region direct ga ivvakunda aws_region variable nunchi value teesukuntundi.

---

14. terraform/terraform.tfvars >> Variable values kosam

```
aws_region    = "ap-south-1"
instance_type = "t3.micro"
server_name   = "netflix-server"
```

explanation:
aws_region = "ap-south-1" > AWS region value
→ aws_region variable ki Mumbai region value istunnam.
instance_type = "t3.micro" > EC2 type value
→ instance_type variable ki t3.micro value istunnam.
server_name = "netflix-server" > server name value
→ server_name variable ki netflix-server value istunnam.

---

15. terraform/main.tf >> Main infrastructure resources kosam

```
resource "aws_instance" "netflix" {
  ami         = "ami-xxxxxxxx"
  instance_type = var.instance_type

  tags = {
    Name = var.server_name
  }
}
```

explanation:
resource "aws_instance" "netflix" > resource
→ AWS lo EC2 instance create cheyyali ani Terraform ki chepthundi.
ami: "ami-xxxxxxxx" > machine image
→ EC2 server ki use cheyyalsina AMI image ni specify chestundi.
instance_type: var.instance_type > server size
→ EC2 instance type direct ga ivvakunda variable nunchi value teesukuntundi.
tags: > resource tags
→ AWS resource ki identifying information add cheyyadaniki use chestham.
Name: var.server_name > server name
→ Server name ni server_name variable nunchi teesukuntundi.